УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 2

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ РАБОТЫ С ЛОГИЧЕСКИМ ТИПОМ ДАННЫХ

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание простейших программ работы с логическим типом данных

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Преобразование типов данных

В языке **Python** используется ***динамическая типизация***. После присваивания значения в переменной сохраняется ссылка на объект определенного типа, а не сам объект. Если затем переменной присвоить значение другого типа, то переменная будет ссылаться на другой объект, и тип данных соответственно изменится. Таким образом, тип данных в языке **Python** — это характеристика объекта, а не переменной. Переменная всегда содержит только ссылку на объект.

После присваивания переменной значения над последним можно производить операции, предназначенные лишь для этого типа данных. Например, строку нельзя сложить с числом, т. к. это приведет к выводу сообщения об ошибке.

Для преобразования типов данных предназначены следующие функции:

- **bool ([<*объект*>])** - преобразует объект в логический тип данных. Примеры:

```
print(bool(0), bool(1), bool(""), bool("Строка"), bool([1, 2]), bool([]))
(False, True, False, True, True, False)
```

- **int ([<*объект*>[, <*система счисления*>]])** - преобразует объект в число. Во втором параметре можно указать систему счисления (значение по умолчанию - 10). Примеры:

```
print(int(7.5), int("71"))
(7, 71)
print(int("71", 10), int("71", 8), int("0o71", 8), int("A", 16))
(71, 57, 57, 10)
```

Если преобразование невозможно, то генерируется исключение:

```
print(int("71s"))
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
```

```
  int("71s")
ValueError: invalid literal for int() with base 10: '71s'
```

– **float ([<*число или строка*>])** - преобразует целое число или строку в вещественное число. Примеры:

```
print(float(7), float("7.1"))
(7.0, 7.1)
print(float("Infinity"), float("-inf"))
(inf, -inf)
print(float("Infinity") + float("-inf"))
nan
```

– **str ([<*объект*>])** - преобразует объект в строку. Примеры:

```
print(str(125), str([1, 2, 3]))
('125', '[1, 2, 3]')
print(str((1, 2, 3)), str({'x': 5, 'y': 10}))
('(1, 2, 3)', "{'y': 10, 'x': 5}")
print(str(bytes('строка', 'utf-8')))
"b'\\xd1\\x81\\xd1\\x82\\xd1\\x80\\xd0\\xbe\\xd0\\xba\\xd0\\xb0'"
print(str(bytearray('строка', 'utf-8')))
"bytearray(b'\\xd1\\x81\\xd1\\x82\\xd1\\x80\\xd0\\xbe\\xd0\\xba\\xd0\\xb0')"
```

– **str (<*объект*> [, <*кодировка*> [, <*обработка ошибок*>]])** - преобразует объект типа **bytes** или **bytearray** в строку. В третьем параметре можно задать значение **"strict"** (при ошибке возбуждается исключение **UnicodeDecodeError** - значение по умолчанию), **"replace"** (неизвестный символ заменяется символом, имеющим код **\uFFFD**) или **"ignore"** (неизвестные символы игнорируются). Примеры:

```
obj1 = bytes("строка1", "utf-8")
obj2 = bytearray("строка2", "utf-8")
print(str(obj1, 'utf-8'), str(obj2, 'utf-8'))
('строка1', 'строка2')
print(str(obj1, "ascii", "ignore"))
'1'
print(str(obj1, "ascii", "strict"))
Traceback (most recent call last):
  File "<pyshell#10>", line 1, in <module>
    str(obj1, "ascii", "strict")
UnicodeDecodeError: 'ascii' codec can't decode byte 0xd1 in position 0:
ordinal not in range(128)
```

– **bytes (<*строка*>, <*кодировка*> [, <*обработка ошибок*>])** - преобразует строку в объект типа **bytes**. В третьем параметре могут быть указаны значения **"strict"** (значение по умолчанию), **"replace"** или **"ignore"**. Примеры:

```
print(bytes('строка', 'cp1251'))
b'\xf1\xf2\xf0\xee\xea\xe0'
print(bytes('строка123', 'ascii', 'ignore'))
b'123'
```

– **bytes (<*последовательность*>)** - преобразует последовательность целых чисел от 0 до 255 в объект типа **bytes**. Если число не попадает в диапазон, то возбуждается исключение **ValueError**. Пример:

```
b = bytes([225, 226, 224, 174, 170, 160])
print(b)
b'\xe1\xe2\xe0\xae\xaa\xa0'
print(str(b, 'cp866'))
'строка'
```

– **bytearray (<*строка*>, <*кодировка*>[, <*обработка ошибок*>])** - преобразует строку в объект типа **bytearray**. В третьем параметре могут быть указаны значения **"strict"** (значение по умолчанию), **"replace"** или **"ignore"**. Пример:

```
print(bytearray('строка', 'cp1251'))
bytearray(b'\xf1\xf2\xf0\xee\xea\xe0')
```

– **bytearray (<*последовательность*>)** - преобразует последовательность целых чисел от 0 до 255 в объект типа **bytearray**. Если число не попадает в диапазон, то возбуждается исключение **ValueError**. Пример:

```
b = bytearray([225, 226, 224, 174, 170, 160])
print(b)
bytearray(b'\xe1\xe2\xe0\xae\xaa\xa0')
print(str(b, 'cp866'))
'строка'
```

– **list (<последовательность>)** - преобразует элементы последовательности в список. Примеры:

```
print(list("12345")) # Преобразование строки
['1', '2', '3', '4', '5']
print(list((1, 2, 3, 4, 5))) # Преобразование кортежа
[1, 2, 3, 4, 5]
```

– **tuple (<последовательность>)** - преобразует элементы последовательности в кортеж:

```
print(tuple("123456")) # Преобразование строки
('1', '2', '3', '4', '5', '6')
print(tuple([1, 2, 3, 4, 5, 6])) # Преобразование списка
(1, 2, 3, 4, 5, 6)
```

В качестве примера рассмотрим возможность сложения двух чисел, введенных пользователем. Как вы уже знаете, вводить данные позволяет функция **input ()**. Воспользуемся этой функцией для получения чисел от пользователя (рисунок 1).

```
# -*- coding: utf-8 -*-
x = input("x = ") # Вводим первое значение
y = input("y = ") # Вводим второе значение
print(x + y)
input()
```

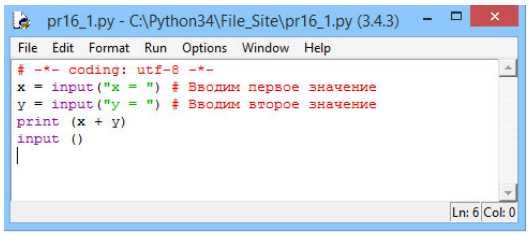


Рисунок 1 - Получение данных от пользователя

Результатом выполнения этого скрипта будет не число, а строка 512 (рисунок 2).

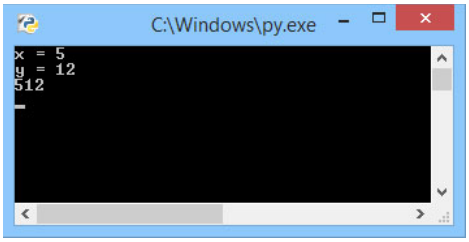


Рисунок 2 - Результат выполнения скрипта

Таким образом, следует запомнить, что функция **input ()** возвращает результат в виде строки. Чтобы просуммировать два числа, необходимо преобразовать строку в число (рисунок 3).

```
# -*- coding: utf-8 -*-
x = int(input("x = ")) # Вводим первое значение
y = int(input("y = ")) # Вводим второе значение
print(x + y)
input()
```

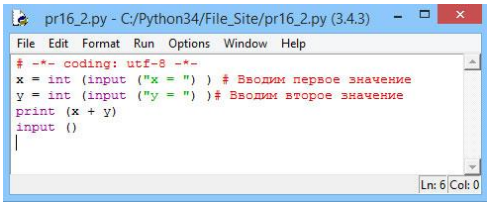


Рисунок 3 - Преобразование строки в число

В этом случае мы получим число 17, как и должно быть (рисунок 4).

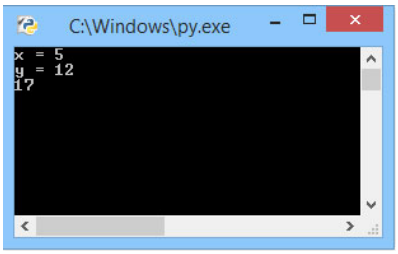


Рисунок 4 Результат выполнения скрипта

Однако если пользователь вместо числа введет строку, то программа завершится с фатальной ошибкой.

4.2 Удаление переменной

Удалить переменную можно с помощью инструкции **del**:

```
del <Переменная1> [, ..., <ПеременнаяN>]
```

Пример удаления одной переменной:

```
x = 10
print(x)
10
x = 10
print(x)
del x
Traceback (most recent call last):
  File "<pyshell#25>", line 1, in <module>
    del x; x
NameError: name 'x' is not defined
```

Пример удаления нескольких переменных:

```
x, y = 10, 20
print(x, y)
del x, y
```

4.3 Особенности использования логических значений

Условные операторы позволяют в зависимости от значения логического выражения выполнить отдельный участок программы или, наоборот, не выполнить его. Логические выражения возвращают только два значения: **True** (**истина**) или **False** (**ложь**), которые ведут себя как целые числа 1 и 0 соответственно:

```
print(True + 2) # Эквивалентно 1 + 2
3
print(False + 2) # Эквивалентно 0 + 2
2
```

Логическое значение можно сохранить в переменной:

```
x = True
y = False
print(x, y)
(True, False)
```

Любой объект в логическом контексте может интерпретироваться как истина (**True**) или как ложь (**False**). Для определения логического значения можно использовать функцию **bool()**.

Значение **True** возвращает следующие объекты:

- любое число, не равное нулю:

```
print(bool(1), bool(20), bool(-20))
```

```
(True, True, True)
print(bool(1.0), bool(0.1), bool(-20.0))
(True, True, True)
```

– не пустой объект:

```
print(bool("0"), bool([0, None]), bool((None, )), bool({'x': 5}))
(True, True, True, True)
```

Следующие объекты интерпретируются как **False**:

– число, равное нулю:

```
print(bool(0), bool(0.0))
(False, False)
```

– пустой объект:

```
print(bool(""), bool([]), bool(()))
(False, False, False)
```

– значение **None**:

```
print(bool(None))
False
```

4.4 Операторы сравнения в языке Python

Операторы сравнения используются в логических выражениях. Перечислим их:

– == (равно):

```
1 == 1, 1 == 5
(True, False)
```

– != (не равно):

```
1 != 5, 1 != 1
(True, False)
```

– < (меньше):

```
1 < 5, 1 < 0
(True, False)
```

– > (больше):

```
1 > 0, 1 > 5
(True, False)
```

– <= (меньше или равно):

```
1 <=5, 1 <=0, 1 <= 1
(True, False, True)
```

– >= (больше или равно):

```
1 >=0, 1 >= 5, 1 >= 1
(True, False, True)
```

– in (проверка на вхождение в последовательность):

```
'Строка' in 'Строка для поиска' # Строки
True
2 in [1, 2, 3], 4 in [1, 2, 3]  # Списки
(True, False)
2 in (1, 2, 3), 4 in (1, 2, 3)  # Кортежи
```

```
(True, False)
```

Оператор in можно также использовать для проверки существования ключа словаря:

```
'x' in {'x': 1, 'y': 2}, 'z' in {'x': 1, 'y': 2}
(True, False)
```

– not in (проверка на невхождение в последовательность):

```
'Строка' not in 'Строка для поиска'     # Строки
False
not in [1, 2, 3], 4 not in [1, 2, 3]  # Списки
(False, True)
2 not in (1, 2, 3), 4 not in (1, 2, 3)  # Кортежи
(False, True)
```

– is - проверяет, ссылаются ли две переменные на один и тот же объект. Если переменные ссылаются на один и тот же объект, то оператор is возвращает значение True:

```
x = y = [1, 2]
x is y
True
x = [1, 2]
y = [1, 2]
x is y
False
```

– is not - проверяет, ссылаются ли две переменные на разные объекты. Если это так, возвращается значение True:

```
x = y = [1, 2]
x is not y
False
x = [1, 2]
y = [1, 2]
x is not y
True
```

Значение логического выражения можно инвертировать с помощью оператора not:

```
x = 1
y = 1
x == y
True
not (x == y), not x == y
(False, False)
```

Если переменные **x** и **y** равны, то возвращается значение **True**, но так как перед выражением стоит оператор **not**, выражение вернет **False**. Круглые скобки можно не указывать, поскольку оператор **not** имеет более низкий приоритет выполнения, чем операторы сравнения.

В логическом выражении можно указывать сразу несколько условий:

```
x = 10
```

```
1 < x < 20, 11 < x < 20
(True, False)
```

Несколько логических выражений можно объединить в одно большое с помощью следующих операторов:

- and (логическое И). Если x в выражении x and y интерпретируется как False, то возвращается x, в противном случае - y:

```
1 < 5 and 2 < 5 # True and True == True
True
1 < 5 and 2 > 5 # True and False == False
False
1 > 5 and 2 < 5 # False and True == False
False
10 and 20, 0 and 20, 10 and 0
(20, 0, 0)
```

- or (логическое ИЛИ). Если x в выражении x or y интерпретируется как False, то возвращается y, в противном случае - x:

```
1 < 5 or 2 < 5 # True or True == True
True
1 < 5 or 2 > 5 # True or False == True
True
1 > 5 or 2 < 5 # False or True == True
True
10 or 20, 0 or 20, 10 or 0
(10, 20, 10)
0 or "" or None or [] or "s"
's'
```

Следующее выражение вернет **True** только в случае, если оба выражения вернут **True**:

```
x1 = x2 and x2 != x3
```

А это выражение вернет **True**, если хотя бы одно из выражений вернет **True**:

```
x1 = x2 or x3 = x4
```

Перечислим операторы сравнения в порядке убывания приоритета:

- <, >, <=, >=, ==, !=, is, is not, in, not in.
- not (логическое отрицание).
- and (логическое И).
- or (логическое ИЛИ).

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Пользователь вводит число. Определите, является ли введённое число чётным.

2 Пользователь вводит два числа. Сравните два введённых числа и выведите True, если первое больше второго, и False — в противном случае.

3 Пользователь вводит число. Выведите True, если оно находится в диапазоне от 10 до 50 включительно.

4 Пользователь вводит число. Определите, является ли оно положительным и нечётным.

5 Пользователь вводит число. Проверьте, является ли введённый год високосным.

6 Пользователь вводит число. Определите, является ли введённое число положительным.

7 Пользователь вводит число. Выведите True, если оно одновременно больше 0 и меньше 100.

8 Пользователь вводит число. Определите, делится ли оно на 3 и на 5 одновременно.

9 Пользователь вводит число. Проверьте, равно ли оно 12345.

10 Пользователь вводит число. Выведите True, если оно положительное и чётное.

11 Пользователь вводит два числа. Выполните логические операции AND, OR, NOT для двух введённых булевых значений (True или False).

12 Пользователь вводит число. Определите, является ли оно кратным 2 или 7.

13 Пользователь вводит три числа — стороны треугольника. Определите, можно ли из них составить треугольник (условие: сумма любых двух сторон больше третьей).

14 Пользователь вводит число. Определите, находится ли оно вне диапазона от 20 до 80 (не включительно).

15 Пользователь вводит три числа. Определите, является ли хотя бы одно из них чётным.

5.2 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Пользователь вводит три числа. Определите, можно ли из них составить прямоугольный треугольник (по теореме Пифагора).

2 Пользователь вводит число. Определите, является ли оно трёхзначным положительным и при этом кратным 7.

3 Пользователь вводит четыре числа. Выведите True, если хотя бы два из них равны между собой.

4 Пользователь вводит два числа. Определите, является ли их сумма чётной и больше 100.

5 Дано число. Определите, является ли оно одновременно простым и меньше 50.

6 Пользователь вводит три числа. Выведите True, если хотя бы одно из них отрицательное, но сумма всех больше нуля.

7 Пользователь вводит число. Определите, можно ли его представить в виде квадрата целого числа или куба целого числа.

8 Даны три числа. Определите, образуют ли они возрастающую или убывающую последовательность.

9 Пользователь вводит два числа. Выведите True, если оба числа имеют одинаковую чётность (оба чётные или оба нечётные).

10 Пользователь вводит число. Определите, является ли оно делителем хотя бы одного из чисел 100, 200 или 300.

11 Даны четыре числа. Проверьте, образуют ли они стороны прямоугольника (противоположные равны).

12 Пользователь вводит число. Определите, находится ли оно одновременно между –100 и –10 или между 10 и 100.

13 Пользователь вводит три числа. Определите, можно ли из них составить равнобедренный или равносторонний треугольник.

14 Пользователь вводит число. Определите, является ли оно одновременно квадратом чётного числа.

15 Пользователь вводит три числа. Определите, образуют ли они арифметическую прогрессию (разность между соседними числами одинакова).

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы

6.2 Цель лабораторной работы

6.3 Индивидуальное задание (текст программы и скриншоты выполнения)

7 Контрольные вопросы

7.1 Опишите операторы сравнения в языке программирования Python.

7.2 Опишите логические операторы в языке программирования Python.

7.3 Охарактеризуйте процесс удаления переменной в языке программирования Python.

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.